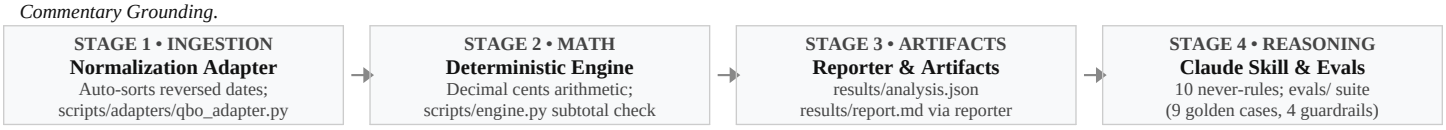
Financial Trend Analysis & Accounting Intelligence Platform

Architecture Note: Implemented Calculation Engine, LLM Grounding, Read-Only MCP Bridge & Production Roadmap

Task 4 Architecture Note • Author: Chandrashekhar H S • Status: Working Prototype (Automated Pipeline) • Verified: 99 Tests, 110/110 Evals

1. Project Structure: Files, Skills & Memory Model

The platform ingests client workbooks or live QuickBooks data and produces validated FP&A trend analyses. The architecture enforces strict separation between deterministic arithmetic and AI reasoning: the Python calculation engine calculates all financial metrics and creates structured outputs (results/analysis.json); the Claude Skill (.claude/skills/monthly-trend-analysis/SKILL.md) uses those results to explain trends, variance drivers, and anomalies without recalculating figures in text. The unified launcher (scripts/run_pipeline.py [--offline]) orchestrates the 6-stage flow: *Pre-flight Safety → Data Normalization → Engine Math → Artifact Verification → Golden Evals → Commentary Grounding.*



Stage	Component & Source File	Deterministic Responsibility & Accounting Method	Verified Artifact & Guarantee
1. Ingestion	`openpyxl / qbo_adapter.py`	Auto-detects and re-orders reversed columns; normalizes QBO V3 P&L JSON into standard 18-month schema.	Chronological period matrix; 16/18 exact matches (2 differ by 3 pre-existing sample txns).
2. Core Math	`scripts/engine.py`	Python Decimal(ROUND_HALF_UP) cents arithmetic. Backward component matching isolates multi-tier subtotals without double-counting.	126/126 checks reconcile (100%); zero floating-point drift; strict 12-consecutive-month TTM gating.
3. Artifacts	`scripts/reporter.py`	Bridges Dec 2025 -$1.244M tax credit to Normalized Core Operating Loss (-$381k). Derives periods dynamically; neutral headings for profitable firms.	analysis.json (contract of truth) & report.md with 10-point CFO close checklist.
4. Reasoning	`Claude Skill (SKILL.md)`	Governed by 10 negative never-rules; grounds strictly on analysis.json without recalculating figures; surfaces anomalies for human controller review.	CFO-ready executive briefing; evaluated by evals/ harness (9 golden cases, 4 guardrails).

2. Credential Security & Execution Environment

Working Prototype (Today): QuickBooks OAuth 2.0 credentials load into process memory from gitignored .env files. The official TypeScript MCP server enforces **registration-time mutation suppression**: at server startup, tool registration logic filters out any tool matching recognized mutation prefixes (create_*, update_*, delete_*), leaving exactly 71 read-only tools registered (out of 142 total). A pre-flight gate in run_pipeline.py halts execution if mutation disable flags are not explicitly true. *Security limitation:* Environment flags suppress tool registration at startup; complete defense-in-depth requires appropriate QuickBooks OAuth app permissions and independent validation of exposed tools.

Production Vault Architecture (Proposed): Secrets migrate to **AWS Secrets Manager** partitioned per tenant (/tenants/{tenant_id}/qbo_credentials) with **AWS KMS** envelope encryption. IAM roles grant access strictly to the authenticated tenant secret. Refresh tokens rotate on each exchange; distributed **Redis** locks serialize refresh across concurrent workers to eliminate race conditions.

3. Hosting Model: Prototype vs. Enterprise Cloud

Current Working Prototype: Operates locally on developer workstation across Python 3.10+ and Node.js 18+. Claude Desktop runs locally, spawning the MCP server via stdio JSON-RPC. Local output files and configuration. QuickBooks sandbox integration verified with balanced journal entries. *Fitness:* Highly effective for development, rapid offline testing, and interactive demonstrations without cloud hosting overhead. Synchronous stdout/stderr telemetry; stateless CLI fails fast with exit code 1; recovery via re-execution.

Proposed Production Architecture (AWS): Supports centralized multi-user operation, access controls, observability, and scaling (proposed design, not deployed):

• **Compute:** Stateless containerized tasks on **AWS ECS Fargate** in private VPC subnets.

• **Ingress: AWS API Gateway** terminates TLS 1.3, verifies OIDC JWTs, and enforces token-bucket rate limits.

• **Queue & Storage: Amazon SQS** buffers jobs; artifacts persist in encrypted **Amazon S3**.

• **Observability:** Centralized JSON logs in **Amazon CloudWatch** with P99 latency and error alarms.

4. Multi-Tenant Isolation Strategy (10 Clients)

Ten-Client Scaling Bottlenecks: The prototype uses single-tenant filesystem paths (results/analysis.json) and a single active .env configuration. At ten concurrent clients, file collisions, OAuth refresh token race conditions, cross-tenant prompt leakage, and Intuit rate limits (500 req/min, 10 concurrent per realm) break single-tenant execution.

Isolation Domain	Prototype (Today)	Proposed 10-Tenant Cloud Blueprint
Tenant Ingress	Single CLI invocation; developer sandbox.	Enterprise OIDC JWT (Okta/Azure AD); Gateway validates token server-side, derives tenant ID; strips client headers.
Data Storage	Local directory; manual path arguments.	Amazon S3 partitioned by s3://bucket/{tenant_id}/analyses/ with scoped IAM session policies.
Database / State	Single-file JSON output; SQLite cache.	PostgreSQL with Row-Level Security (RLS) enforcing tenant_id = current_setting('app.current_tenant') (proposed).
Secrets & Tokens	Single .env file on workstation.	AWS Secrets Manager: /tenants/{tenant_id}/qbo_credentials; isolated IAM roles; token refresh concurrency locks.
LLM Context	Claude Desktop local session context.	Ephemeral, stateless agent tasks; loads only client workbook & analysis.json; zero shared prompt/cache context.
Operations	Synchronous console output; fail-fast.	Per-tenant rate limits & DLQ queues; tenant-safe correlation IDs; zero financial/token logging; failure isolation.

Memory Model Contract: The prototype retains no persistent cross-session conversational memory by design. Analysis state is stored in structured artifacts (analysis.json, report.md) and configuration. Client-specific folders provide local operational separation during development, but local folder segregation alone does NOT provide enterprise security isolation.

5. Architectural Verification & Compliance Sign-Off

Verified Implementation Metrics (Prototype Today): • **Pytest Test Suite: 99 passed** in 18.54s (100% pass rate across engine, CLI, mutation, adapter, and reconciliation suites). • **Golden Evaluation Harness: 110.0 / 110.0** (100.0% score across 9 golden cases, +0.0 delta vs. baseline, zero regressions). • **Commentary Evaluation Suite: 100.0 / 100.0** across all 4 guardrail categories (Numerical Grounding, Unsupported Causal Claims, Anomaly Interpretation, Missing Information Disclosure). • **QBO Ingestion Stretch Goal:** QuickBooks sandbox integration verified; tenant-specific identifiers are kept out of documentation. 18 balanced journal entries seeded; 16/18 months match Excel baseline exactly. **Scope Boundary:** This document certifies the working prototype implementation and specifies the enterprise AWS multi-tenant cloud roadmap. Proposed cloud infrastructure is planned, not deployed.